Technology Stack

Date	01/07/2026
Team ID	
Project Name	Smart Lender
Maximum Marks	2 Marks

Define Your Technology Stack:

The table below identifies the programming languages, frameworks, storage, hosting, version control, and supporting libraries used to build the Smart Lender loan-approval prediction system, based on its current Flask application and training notebook.

Technology Stack Details

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	**Frontend / Client-Side** *(e.g., React, Angular, HTML/CSS)*	HTML5, CSS3, Jinja2 templating (via Flask)	Server-rendered pages (home.html, predict.html, submit.html) keep the UI simple and lightweight; no build step or JS framework is needed for a straightforward form-and-result workflow. style.css provides a clean, responsive layout.
2	**Backend / Server-Side** *(e.g., Node.js, Python/Django, Java)*	Python 3 with Flask (app.py)	Flask is a lightweight micro-framework that integrates naturally with Python's ML ecosystem (scikit-learn, pandas, numpy), keeping routing (/, /predict, /submit) and model-inference logic in the same language used for training.
3	**Database / Data Storage** *(e.g., MySQL, MongoDB, PostgreSQL)*	File-based storage - CSV/XLSX dataset + Pickle (.pkl) model artifacts	The historical loan dataset (loan_prediction.csv/.xlsx) is static and used only for training, so no relational/NoSQL database is required. The trained Random Forest model (rdf.pkl) and StandardScaler (scale1.pkl) are serialized with pickle and loaded once at app startup for fast, dependency-free inference.
4	**Cloud / Hosting / Deployment** *(e.g., AWS, Azure, Heroku, Docker)*	Render (Gunicorn WSGI server)	app.py reads the PORT from the environment and binds to 0.0.0.0, and requirements.txt includes gunicorn - the standard pattern for deploying a Flask app on Render. Render offers easy, cost-effective Git-based deployment suited to a small ML web app.
5	**Version Control & CI/CD** *(e.g., GitHub, GitLab, Jenkins)*	GitHub	The project is organized as a GitHub repository (Smart-Lender-main) for source control and collaboration. No automated CI/CD pipeline is configured yet; deployment to Render is triggered manually / via Git push.
6	**Third-Party APIs / Other Tools** *(e.g., Stripe, Twilio, Google Maps)*	scikit-learn, pandas, NumPy, imbalanced-learn, Jupyter Notebook	No external third-party APIs are used. scikit-learn provides the ML algorithms (Random Forest, Decision Tree, KNN, Gradient Boosting were compared) and StandardScaler used for preprocessing; pandas/NumPy handle data manipulation; imbalanced-learn supports handling class imbalance in the loan dataset; the Training notebook documents and reproduces model development.